RETAIL SALES DATA WAREHOUSE

SQL Analysis Project · Microsoft SQL Server · Gold Schema

7	70+	3	6
Analysis Sections	SQL Queries	Core Tables	Query Categories

Author: Deepak | GitHub: github.com/deepak015/-SQL-Sales-Analysis-Project Database: Microsoft SQL Server | Date: May 2026

■ Project Overview

This document presents a complete SQL-based analysis of a Retail Sales Data Warehouse built on Microsoft SQL Server. All queries operate on the **gold** schema — the curated, business-ready layer of the warehouse following the Medallion Architecture (Bronze → Silver → Gold).

The project covers customer segmentation, product performance, sales trends, date/time analysis, and advanced window-function analytics — totalling 70+ production-ready queries across 6 topic areas.

Database Schema

Table	Type	Key Columns	Description
gold.dim_customers	Dimension	customer_id, customer_key	Customer demographics & geography
gold.dim_products	Dimension	product_id, product_key	Product catalog with cost & category
gold.fact_sales	Fact	order_number, customer_key, product_key	Transactional sales records

Contents

■ Customer Dimension (gold.dim_customers)

Q01 Total number of customers

```
SELECT COUNT(*) AS Total_Customers
FROM gold.dim_customers
```

Q02 All unique customer countries

```
SELECT DISTINCT country
FROM gold.dim_customers
```

Q03 Number of customers per country

```
SELECT country, COUNT(*) AS Total_Customers
FROM gold.dim_customers
GROUP BY country
```

Q04 Male and female customer count

```
SELECT gender, COUNT(*) AS Total_Customers
FROM gold.dim_customers
GROUP BY gender
```

Q05 Married vs single customer count

```
SELECT marital_status, COUNT(*) AS Total_Customers
FROM gold.dim_customers
GROUP BY marital_status
```

Q06 Top 10 oldest customers

```
SELECT TOP 10 customer_id, first_name, last_name, birthdate
FROM gold.dim_customers
WHERE birthdate IS NOT NULL
ORDER BY birthdate ASC
```

Q07 Customers born after 1968

```
SELECT customer_id, first_name, last_name, birthdate

FROM gold.dim_customers

WHERE birthdate > '1968'
```

Q08 Average customer age

```
SELECT first_name, last_name,

AVG(DATEDIFF(year, birthdate, GETDATE())) AS Average_Age

FROM gold.dim_customers

WHERE birthdate IS NOT NULL

GROUP BY first_name, last_name
```

Q09 Customers whose first name starts with A

```
SELECT first_name

FROM gold.dim_customers

WHERE first_name LIKE 'A%'
```

Q10 Country with maximum customers

```
SELECT TOP 1 country, COUNT(*) AS total_customers

FROM gold.dim_customers

GROUP BY country

ORDER BY COUNT(*) DESC
```

■ Product Dimension (gold.dim_products)

Q01 Total number of products

```
SELECT COUNT(*) AS TotalProducts

FROM gold.dim_products
```

Q02 Total distinct product categories

```
SELECT COUNT(DISTINCT category) AS TotalCategories

FROM gold.dim_products
```

Q03 Number of products per category

```
SELECT category, COUNT(*) AS ProductsInCategory

FROM gold.dim_products

GROUP BY category
```

Q04 Most expensive product

```
SELECT TOP 1 product_name, cost

FROM gold.dim_products

ORDER BY cost DESC
```

Q05 Least expensive product

```
SELECT TOP 1 product_name, cost

FROM gold.dim_products

ORDER BY cost ASC
```

Q06 Products that require maintenance

```
SELECT product_name, maintenance

FROM gold.dim_products

WHERE maintenance = 'Yes'
```

Q07 Products launched after 2010

```
SELECT product_name, start_date

FROM gold.dim_products

WHERE start_date > '2010-12-31'
```

Q08 Total products per product line

```
SELECT product_line, COUNT(*) AS TotalProductsInLine

FROM gold.dim_products

GROUP BY product_line
```

Q09 Average product cost

```
SELECT AVG(cost) AS AverageProductCost

FROM gold.dim_products
```

Q10 Category with highest number of products

```
SELECT TOP 1 category, COUNT(*) AS ProductsInCategory

FROM gold.dim_products

GROUP BY category

ORDER BY ProductsInCategory DESC
```

■ Sales Fact Table (gold.fact_sales)

Q01 Total sales amount

```
SELECT SUM(sales_amount) AS total_sales_amount

FROM gold.fact_sales
```

Q02 Total quantity sold

```
SELECT SUM(quantity) AS total_quantity_sold

FROM gold.fact_sales
```

Q03 Total number of orders

```
SELECT COUNT(DISTINCT order_number) AS total_orders

FROM gold.fact_sales
```

Q04 Average sales amount

```
SELECT AVG(sales_amount) AS average_sales_amount

FROM gold.fact_sales
```

Q05 Highest sales amount

```
SELECT MAX(sales_amount) AS highest_sales_amount

FROM gold.fact_sales
```

Q06 Lowest sales amount

```
SELECT MIN(sales_amount) AS lowest_sales_amount

FROM gold.fact_sales
```

Q07 Monthly sales trend

```
SELECT YEAR(order_date) AS sales_year,
MONTH(order_date) AS sales_month,
SUM(sales_amount) AS monthly_sales
FROM gold.fact_sales
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY sales_year, sales_month
```

Q08 Yearly sales trend

```
SELECT YEAR(order_date) AS sales_year,
SUM(sales_amount) AS yearly_sales
FROM gold.fact_sales
GROUP BY YEAR(order_date)
ORDER BY sales_year
```

Q09 Sales for each calendar month

```
SELECT MONTH(order_date) AS sales_month,
SUM(sales_amount) AS monthly_sales
FROM gold.fact_sales
GROUP BY MONTH(order_date)
ORDER BY sales_month
```

Q10 Busiest sales month

```
SELECT TOP 1 MONTH(order_date) AS sales_month,
SUM(sales_amount) AS monthly_sales
FROM gold.fact_sales
GROUP BY MONTH(order_date)
ORDER BY monthly_sales DESC
```

■ Customer + Orders Analysis

Q01 Top 10 customers by total spending

```
SELECT TOP 10 c.customer_id, c.first_name, c.last_name,
SUM(o.sales_amount) AS total_spending
FROM gold.dim_customers c
LEFT JOIN gold.fact_sales o ON c.customer_id = o.customer_key
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_spending DESC
```

Q02 Customer who placed maximum orders

```
SELECT TOP 1 c.customer_id, c.first_name,
COUNT(o.order_number) AS max_orders
FROM gold.dim_customers c
LEFT JOIN gold.fact_sales o ON c.customer_id = o.customer_key
GROUP BY c.customer_id, c.first_name
ORDER BY max_orders DESC
```

Q03 Average revenue per customer

```
SELECT c.customer_id, c.first_name,
AVG(o.sales_amount) AS avg_revenue
FROM gold.dim_customers c
LEFT JOIN gold.fact_sales o ON c.customer_id = o.customer_key
GROUP BY c.customer_id, c.first_name
ORDER BY avg_revenue DESC
```

Q04 Country-wise sales revenue

```
SELECT c.country, SUM(o.sales_amount) AS country_sales_revenue

FROM gold.dim_customers c

LEFT JOIN gold.fact_sales o ON c.customer_id = o.customer_key

GROUP BY c.country

ORDER BY country_sales_revenue DESC
```

Q05 Gender-wise sales analysis

```
SELECT c.gender, SUM(o.sales_amount) AS gender_wise_sales

FROM gold.dim_customers c

LEFT JOIN gold.fact_sales o ON c.customer_id = o.customer_key

GROUP BY c.gender

ORDER BY gender_wise_sales DESC
```

Q06 Married vs single customer sales

```
SELECT c.marital_status,

SUM(o.sales_amount) AS marital_status_sales

FROM gold.dim_customers c

LEFT JOIN gold.fact_sales o ON c.customer_id = o.customer_key

GROUP BY c.marital_status

ORDER BY marital_status_sales DESC
```

Q07 Customer who purchased most quantity

```
SELECT TOP 1 c.customer_id, c.first_name,

SUM(o.quantity) AS total_quantity

FROM gold.dim_customers c

LEFT JOIN gold.fact_sales o ON c.customer_id = o.customer_key

GROUP BY c.customer_id, c.first_name

ORDER BY total_quantity DESC
```

Q08 Top spending female customer

```
SELECT TOP 1 c.customer_key, c.first_name, c.gender,
SUM(o.sales_amount) AS total_spending
FROM gold.dim_customers c
LEFT JOIN gold.fact_sales o ON c.customer_id = o.customer_key
WHERE c.gender = 'female'
GROUP BY c.customer_key, c.gender, c.first_name
ORDER BY total_spending DESC
```

Q09 Top 5 customers with more than 5 orders

```
SELECT TOP 5 c.customer_id, c.first_name,
COUNT(o.order_number) AS total_orders
FROM gold.dim_customers c
LEFT JOIN gold.fact_sales o ON c.customer_id = o.customer_key
GROUP BY c.customer_id, c.first_name
ORDER BY total_orders DESC
```

Q10 Customer contribution % in total sales

```
SELECT c.customer_id, c.first_name,
SUM(o.sales_amount) AS customer_sales,
ROUND(SUM(o.sales_amount) * 100.0
/ SUM(SUM(o.sales_amount)) OVER (), 2)
AS contribution_percentage
FROM gold.dim_customers c
LEFT JOIN gold.fact_sales o ON c.customer_id = o.customer_key
GROUP BY c.customer_id, c.first_name
ORDER BY contribution_percentage DESC
```

■ Product + Orders Analysis

Q01 Top 10 best-selling products (by quantity)

```
SELECT TOP 10 p.product_id, p.product_name,

SUM(o.quantity) AS best_selling_product

FROM gold.dim_products p

LEFT JOIN gold.fact_sales o ON p.product_key = o.product_key

GROUP BY p.product_id, p.product_name

ORDER BY best_selling_product DESC
```

Q02 Top 10 least-selling products

```
SELECT TOP 10 p.product_id, p.product_name,

SUM(o.quantity) AS least_selling_product

FROM gold.dim_products p

LEFT JOIN gold.fact_sales o ON p.product_key = o.product_key

GROUP BY p.product_id, p.product_name

ORDER BY least_selling_product ASC
```

Q03 Category-wise revenue

```
SELECT p.category, SUM(o.sales_amount) AS category_wise_revenue

FROM gold.dim_products p

LEFT JOIN gold.fact_sales o ON p.product_key = o.product_key

GROUP BY p.category

ORDER BY category_wise_revenue DESC
```

Q04 Subcategory-wise revenue

```
SELECT p.subcategory,

SUM(o.sales_amount) AS category_wise_revenue

FROM gold.dim_products p

LEFT JOIN gold.fact_sales o ON p.product_key = o.product_key

GROUP BY p.subcategory

ORDER BY category_wise_revenue DESC
```

Q05 Total quantity sold per product

```
SELECT p.product_id, p.product_name,

SUM(o.quantity) AS total_quantity_sold

FROM gold.dim_products p

LEFT JOIN gold.fact_sales o ON p.product_key = o.product_key

GROUP BY p.product_id, p.product_name

ORDER BY total_quantity_sold DESC
```

Q06 Highest revenue generating category

```
SELECT TOP 1 p.category,

SUM(o.sales_amount) AS category_wise_revenue

FROM gold.dim_products p

LEFT JOIN gold.fact_sales o ON p.product_key = o.product_key

GROUP BY p.category

ORDER BY category_wise_revenue DESC
```

Q07 Top 5 products by sales amount

```
SELECT TOP 5 p.product_name,

SUM(o.sales_amount) AS total_sales_amount

FROM gold.dim_products p

LEFT JOIN gold.fact_sales o ON p.product_key = o.product_key

GROUP BY p.product_name

ORDER BY total_sales_amount DESC
```

Q08 Products never sold

```
SELECT p.product_name

FROM gold.dim_products p

LEFT JOIN gold.fact_sales o ON p.product_key = o.product_key

WHERE o.product_key IS NULL
```

Q09 Average sales per product

```
SELECT p.product_name,

AVG(o.sales_amount) AS average_sales_per_product

FROM gold.dim_products p

LEFT JOIN gold.fact_sales o ON p.product_key = o.product_key

GROUP BY p.product_name

ORDER BY average_sales_per_product DESC
```

Q10 Category contribution % in total sales

```
SELECT p.category, SUM(o.sales_amount) AS category_sales,

SUM(o.sales_amount) * 100.0

/ (SELECT SUM(sales_amount) FROM gold.fact_sales)

AS percentage_contribution

FROM gold.dim_products p

LEFT JOIN gold.fact_sales o ON p.product_key = o.product_key

GROUP BY p.category

ORDER BY percentage_contribution DESC
```

■ Date & Time Analysis

Q01 **Total sales year-wise**

```
SELECT YEAR(order_date) AS SalesYear,
SUM(sales_amount) AS TotalSales
FROM gold.fact_sales
GROUP BY YEAR(order_date)
ORDER BY SalesYear
```

Q02 **Total sales quarter-wise**

```
SELECT YEAR(order_date) AS SalesYear,
DATEPART(QUARTER, order_date) AS SalesQuarter,
SUM(sales_amount) AS TotalSales
FROM gold.fact_sales
GROUP BY YEAR(order_date), DATEPART(QUARTER, order_date)
ORDER BY SalesYear, SalesQuarter
```

Q03 **Total sales month-wise (with year)**

```
SELECT YEAR(order_date) AS SalesYear,
MONTH(order_date) AS SalesMonth,
SUM(sales_amount) AS TotalSales
FROM gold.fact_sales
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY SalesYear, SalesMonth
```

Q04 Orders placed on weekends

```
SELECT YEAR(order_date) AS SalesYear,

DATEPART(WEEKDAY, order_date) AS Weekday,

SUM(sales_amount) AS TotalSales

FROM gold.fact_sales

WHERE DATEPART(WEEKDAY, order_date) IN (1, 7)

GROUP BY YEAR(order_date), DATEPART(WEEKDAY, order_date)

ORDER BY SalesYear, Weekday
```

Q05 Orders placed in December

```
SELECT YEAR(order_date) AS SalesYear,

MONTH(order_date) AS SalesMonth,

SUM(sales_amount) AS TotalSales

FROM gold.fact_sales

WHERE MONTH(order_date) = 12

GROUP BY YEAR(order_date), MONTH(order_date)

ORDER BY SalesYear, SalesMonth
```

Q06 First order date

```
SELECT MIN(order_date) AS FirstOrderDate

FROM gold.fact_sales
```

Q07 Latest order date

```
SELECT MAX(order_date) AS LatestOrderDate

FROM gold.fact_sales
```

Q08 Average sales per month (year-month)

```
SELECT YEAR(order_date) AS SalesYear,
MONTH(order_date) AS SalesMonth,
AVG(sales_amount) AS AverageSales
FROM gold.fact_sales
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY SalesYear, SalesMonth
```

Q09 Year-over-year sales growth

```
WITH YearlySales AS (
SELECT YEAR(order_date) AS SalesYear,
SUM(sales_amount) AS TotalSales
FROM gold.fact_sales
GROUP BY YEAR(order_date)
)
SELECT SalesYear, TotalSales,
LAG(TotalSales) OVER (ORDER BY SalesYear) AS PrevYearSales,
TotalSales - LAG(TotalSales) OVER (ORDER BY SalesYear)
AS SalesGrowth,
ROUND(
(TotalSales - LAG(TotalSales) OVER (ORDER BY SalesYear))
* 100.0 / LAG(TotalSales) OVER (ORDER BY SalesYear), 2)
AS SalesGrowthPercentage
FROM YearlySales
```

Q10 Running total of sales

```
SELECT order_date, sales_amount,
SUM(sales_amount) OVER (ORDER BY order_date)
AS RunningTotal
FROM gold.fact_sales
```

■ Window Functions (Advanced Analytics)

Q01 **Rank customers by total spending**

```
SELECT customer_id, first_name,

SUM(sales_amount) AS total_spending,

RANK() OVER (ORDER BY SUM(sales_amount) DESC)

AS spending_rank

FROM gold.fact_sales fs

LEFT JOIN gold.dim_customers dc

ON fs.customer_key = dc.customer_key

GROUP BY customer_id, first_name
```

Q02 **Rank products by revenue**

```
SELECT product_id, product_name,

SUM(sales_amount) AS total_revenue,

RANK() OVER (ORDER BY SUM(sales_amount) DESC)

AS revenue_rank

FROM gold.fact_sales fs

LEFT JOIN gold.dim_products dp

ON fs.product_key = dp.product_key

GROUP BY product_id, product_name
```

Q03 Previous month sales using LAG()

```
WITH PreviousMonthSales AS (
SELECT YEAR(order_date) AS order_year,
MONTH(order_date) AS order_month,
SUM(sales_amount) AS total_sales
FROM gold.fact_sales
GROUP BY YEAR(order_date), MONTH(order_date)
)
SELECT order_year, order_month, total_sales,
LAG(total_sales) OVER
(ORDER BY order_year, order_month)
AS previous_month_sales
FROM PreviousMonthSales
```

Q04 Next month sales using LEAD()

```
WITH NextMonthSales AS (
SELECT YEAR(order_date) AS order_year,
MONTH(order_date) AS order_month,
SUM(sales_amount) AS total_sales
FROM gold.fact_sales
GROUP BY YEAR(order_date), MONTH(order_date)
)
SELECT order_year, order_month, total_sales,
LEAD(total_sales) OVER
(ORDER BY order_year, order_month)
AS next_month_sales
FROM NextMonthSales
```

Q05 Top 3 products per category

```
WITH RankedProducts AS (
SELECT p.product_id, p.category, p.product_name,
SUM(o.sales_amount) AS total_revenue
FROM gold.fact_sales o
LEFT JOIN gold.dim_products p ON o.product_key = p.product_key
GROUP BY p.product_id, p.category, p.product_name
),
products_with_rank AS (
SELECT *, RANK() OVER
(PARTITION BY category ORDER BY total_revenue DESC)
AS category_rank
FROM RankedProducts
)
SELECT * FROM products_with_rank WHERE category_rank &lt;= 3
```

Q06 Dense rank of countries by sales

```
WITH CountrySales AS (
SELECT c.country, SUM(o.sales_amount) AS total_sales
FROM gold.fact_sales o
LEFT JOIN gold.dim_customers c ON o.customer_key = c.customer_key
GROUP BY c.country
)
SELECT country, total_sales,
DENSE_RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
FROM CountrySales
```

Q07 Cumulative sales using SUM() OVER()

```
SELECT order_date,
SUM(sales_amount) AS daily_sales,
SUM(SUM(sales_amount)) OVER (ORDER BY order_date)
AS cumulative_sales
FROM gold.fact_sales
GROUP BY order_date
```

Q08 3-month moving average of sales

```
WITH DailySales AS (
SELECT YEAR(order_date) AS order_year,
MONTH(order_date) AS order_month,
SUM(sales_amount) AS total_sales
FROM gold.fact_sales
GROUP BY YEAR(order_date), MONTH(order_date)
)
SELECT order_year, order_month, total_sales,
AVG(total_sales) OVER
(ORDER BY order_year, order_month
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
AS moving_average
FROM DailySales
```

Q09 Customer order sequence number

```
SELECT c.customer_id, fs.order_number,
RANK() OVER
(PARTITION BY c.customer_id ORDER BY fs.order_date)
AS order_sequence
FROM gold.fact_sales fs
LEFT JOIN gold.dim_customers c ON fs.customer_key = c.customer_key
GROUP BY c.customer_id, fs.order_number, fs.order_date
```

Q10 Highest selling product per year

```
WITH YearlyProductSales AS (

SELECT YEAR(order_date) AS order_year,

product_key,

SUM(sales_amount) AS highest_sales

FROM gold.fact_sales

GROUP BY YEAR(order_date), product_key

),

RankedYearlySales AS (

SELECT *, RANK() OVER

(PARTITION BY order_year ORDER BY highest_sales DESC)

AS sales_rank

FROM YearlyProductSales

)

SELECT * FROM RankedYearlySales WHERE sales_rank = 1
```

End of Document

github.com/deepak015/-SQL-Sales-Analysis-Project